

Deep learning and representation

AI ENGINEERING: GENAI AND AGENTIC SYSTEMS

Today you build Project 2, `nw/semantic`: a transformer encoder fine-tuned with LoRA for tags and priority at once, a training loop you can read, an ONNX export with parity proof and an int8 variant, the benchmark that says exactly what the extra cost buys over Project 1, and the same loop around it: a versioned candidate, a gate that reads the export and the benchmark, a card, and three drift signals in the service.

Where we are

PART 1

Foundations and
the AI service layer

PART 2

Machine learning in
production

PART 3

Deep learning and
representation

PART 4

Generative AI:
retrieval and
evaluation

PART 5

Agentic AI systems

PART 6

Deployment,
observability and
capstone

Project 2 is measured against Project 1 on the same test split, the same machine, and promoted through the same shape of gate. The retrieval project reuses its embedding model; the agent calls its service as a tool; the capstone routes between the two on the numbers you produce today.

Today, step by step

- 1 Device, memory arithmetic, the data contract
- 2 Read the loop and see the tests fail
- 3 Freeze the encoder, inject LoRA
- 4 The loss and the training step
- 5 Per-tag thresholds, then train the real model
- 6 Track and compare runs
- 7 Checkpoints that resume **OPTIONAL**
- 8 Export to ONNX, quantise, prove parity
- 9 The benchmark table
- 10 The gate moves latest; read the card
- 11 The service on latest, the third drift signal
OPTIONAL
- 12 Backtest the next version **OPTIONAL**
- 13 The retraining workflow **OPTIONAL**
- 14 Error analysis on the German tickets
OPTIONAL

The tests download nothing: a tiny random DistilBERT, a 600 ticket fixture, seconds on CPU. Step 2 prints `2 failed, 8 passed, 18 errors` ; eight pass before you write a line.

"It does not fit" is usually the optimiser, not the model

Held per parameter, fp32 with Adam	Bytes
Weights	4
Gradients	4
Two optimiser moments	8
Full fine-tuning	16

about 1 GB

66 million DistilBERT parameters at 16 bytes each, before a single activation

Activations scale with batch size times sequence length times hidden size times layers. Every number here is derivable; write it down before you touch a model.

Freeze the encoder and train under one percent of it

```
for p in model.encoder.parameters():
    p.requires_grad = False
cfg = LoraConfig(r=8, lora_alpha=16,
                 lora_dropout=0.05,
                 target_modules=["q_lin", "v_lin"],
                 bias="none")
model.encoder = get_peft_model(model.encoder, cfg)
```

Knob	Effect
Rank <code>r</code>	Adapter capacity. 8 for classification, 64 for generation
<code>alpha</code>	Scale of the adapter's contribution. Convention: 2 times <code>r</code>
Targets	Which linear layers get adapters. Attention query and value is the standard start
Dropout	Regularises the adapter; 0.05 by default

Small low-rank matrices on the query and value projections. Only those and the two heads train: 192,058 of 66,554,938 parameters, 0.29 percent. The optimiser state becomes negligible, and a laptop CPU is enough.

Multi-label is not multi-class

	Tags	Priority
Can two be true at once?	Yes: "SSO" and "Outage"	No
Loss	Binary cross-entropy with logits, per tag	Cross-entropy with class weights
Decision	One threshold per tag, chosen on validation	Argmax
Wrong choice looks like	Softmax across tags: exactly one fires	A flat 0.5 under-predicts every rare tag

`loss_fn` adds the two. Priority is the triage project's imbalance lesson in a different loss, without the threshold rule: this head does not protect P0, and the benchmark will show it.

Five lines per batch, and a finite check that names the step

- 1 Move `input_ids`, `attention_mask`, `tags` and `priority` to the device
- 2 Forward under `torch.autocast`, enabled only on CUDA; divide the loss by `accumulate` inside it
- 3 `scaler.scale(loss).backward()`
- 4 Every `accumulate` batches: unscale, clip to 1.0, step, update the scaler, zero the gradients, step the schedule
- 5 If the loss is not finite, raise with the step number. A NaN at step 40 is a learning rate problem and the message should say so

If the loss never moves: you divided by `accumulate` after backward instead of before, or `zero_grad` runs before the backward instead of after the step.

Training ends with a candidate, not a decision

0.742

P0 recall on the `test:` line. 0.839 on the last validation line. Same model, same argmax, 23 and 26 of 31

- DistilBERT from the Hub, 2,000 ticket stratified subset, six epochs, 102 s on Apple silicon
- Last validation line: tag micro-F1 0.550, priority macro-F1 0.659, P0 recall 0.839
- Then the best epoch reloaded and scored on test: 0.541, 0.693, 0.742
- Then `candidate <version> registered, not promoted`: the gate needs two files that do not exist yet

The artifact is `artifacts/semantic/<version>/`, named like Project 1's: second, git SHA, data hash. Inside: the checkpoints, the thresholds, the metadata with both evaluations by language, the data profile, the card. No `latest`. If 20 steps took longer than a minute, switch to Colab now and rejoin at step 9.

A run is not a promotion, here either

version	subset	epochs	lr	r	tag micro- F1	tag macro- F1	priority macro- F1	P0 recall	s	device
2026...-308ad1151d5c	2000	6	1e-03	8	0.541	0.609	0.693	0.742	102	mps

`make runs-semantic` reads `artifacts/semantic/runs.jsonl`: the eight parameters that matter, every epoch's validation metrics, the test metrics of the best epoch, the seconds and the device. The same run sits in MLflow as the experiment `semantic` and the registered model `northwind-semantic`, alias `candidate` and no `production` yet.

Open the `val_tag_micro_f1` chart: 0.27, 0.40, 0.49, 0.52, 0.54, 0.55, still rising at epoch six. Someone will want twelve. The runs table is where that argument is settled, on the test column, after the benchmark says what each point costs in latency.

ONNX is the serving boundary: the classifier never touches PyTorch again

- 1 Wrap the model, tokenise one short sample, `torch.onnx.export` with named inputs and outputs
- 2 Dynamic axes on batch and sequence, opset 17, `dynamo=False`
- 3 Parity on 20 held-out tickets: fp32 1.0e-5 maximum absolute logit difference; the gate holds it to 1e-4
- 4 Dynamic int8 quantisation: 265.7 MB to 66.8 MB, no calibration set, judged by task metrics, not logits

Where ONNX leaks

Custom operators, data-dependent control flow, and anything the tracer turns into a constant. Read the export warnings once.

The graphs, the tokenizer and `export_report.json` land in the version directory, where the gate reads them.

Three points of macro-F1 cost fifteen times the latency

Model	Priority macro - F1	P0 recall	Tag micro - F1	p50 ms	p95 ms	Size MB
Project 1: TF-IDF and logistic regression	0.663	0.903	n/a	1.7	2.3	6.8
Project 2: PyTorch fp32	0.693	0.742	0.541	28.0	53.3	268.0
Project 2: ONNX fp32	0.693	0.742	0.541	35.1	57.7	265.7
Project 2: ONNX int8	0.682	0.677	0.522	12.5	18.2	66.8

The subset checkpoint, six epochs, Apple silicon, test split, batch size one. Remeasured with the locked dependencies the int8 row reads 0.666, 0.645, 0.524 at 9.6 and 13.5 ms; the gate quotes that run. Yours will differ; the shape will not.

Read the table as a customer would

P0 recall is lower

0.742 against 0.903. Project 1's threshold rule protects P0; argmax does not. The bigger model is not the safer one for the rare class.

int8 halves the latency

28.0 to 12.5 ms at p50, a quarter of the size, and it pays in the rare class first: P0 recall 0.742 to 0.677, then 0.645 on the remeasure. Three tickets.

Project 1 stays in front

The capstone keeps the linear model as the cheap pre-filter and sends the transformer the tickets where tags earn their keep.

A candidate is served only by passing a gate that reads three files

Bar	Value	Yours
Tag micro-F1, test split	at least 0.50	0.541
P0 recall, test split	at least 0.70	0.742
fp32 ONNX parity	at most 1e-4	1.0e-5
int8 against fp32, either head	drop at most 0.03	0.027, 0.017
int8 p95 latency	at most 100 ms	13.5
Against production: tag micro-F1, priority macro-F1	drop at most 0.02	0.000, 0.000
Same test split	identical data hash	308ad1151d5c

A missing file is named, not guessed. Pass: `latest`, the alias, the summary, the card, one line in `promotions.jsonl`. `--force` is recorded.

The card says what the table did not

0 of 2

German P0 tickets caught on the test split. 39 rows: tag micro-F1 0.456 against 0.545, priority macro-F1 0.298 against 0.711

`MODEL_CARD.md` is generated from the artifact's own files and regenerated by the gate, so it cannot go stale:

- intended use, and the sentence that this head does not decide an SLA
- the adapters, the ratio, the data hash, the subset, the seconds, the device
- validation and test side by side, then the same numbers per language
- the tag thresholds: 52 of 54 differ from 0.5, the ten lowest named
- the export, the benchmark, the training data, limitations, monitoring

The service serves latest and knows when the world changes

4.15

tag rate PSI after fifty forty-character tickets. Text length 8.87, the same as Project 1 on the same profile; priority 5.08; level alert

- `NW_SEMANTIC_ARTIFACT=artifacts/semantic/latest`; the root means the same thing, so nothing that failed the gate is served by accident
- Text length and priority reuse Project 1's monitor; the tag rate is this model's own: requests carrying each tag against the training rows' `tag_share`
- `nw_semantic_drift_psi{feature}`,
`nw_semantic_drift_level`, one `drift_alert` line past 0.2, the line the capstone alarms on

A forty-character ticket fires fifteen to twenty of the 54 tags; the tag rate says so first. `NW_SEMANTIC_CAPTURE` keeps the requests for the backtest.

The next version runs beside the served one, and a workflow never promotes

Two epochs, `--no-promote`, 35 seconds. Test P0 recall 0.806, higher than the served 0.742.

Someone sorting the runs table on that column would promote it.

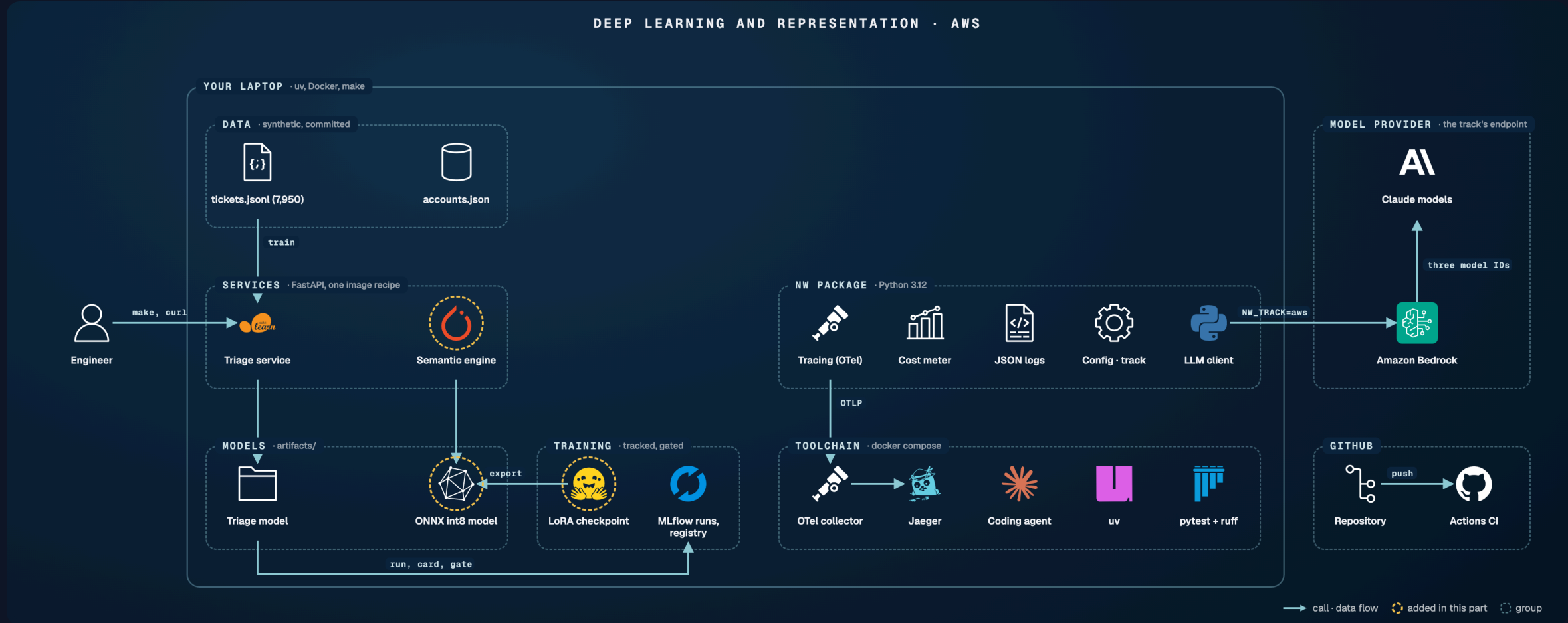
`make backtest-semantic`, int8 as served, 794 tickets: priority agreement 0.728, tag set agreement 0.035, tag Jaccard 0.611. `make promote-semantic`: `GATE FAILED`, tag micro-F1 0.375 below the bar 0.50, and both relative bars.

`retrain-semantic.yml`

On demand only: a hosted CPU runner takes five to eight minutes per epoch, so two epochs prove the loop in 10 to 20 minutes and fail the gate by design.

`data_check`, a Project 1 candidate for the benchmark, `train --no-promote`, `export`, `benchmark`, the gate against `data/golden/semantic_production.json`, the card in the job summary, the candidate uploaded for 30 days.

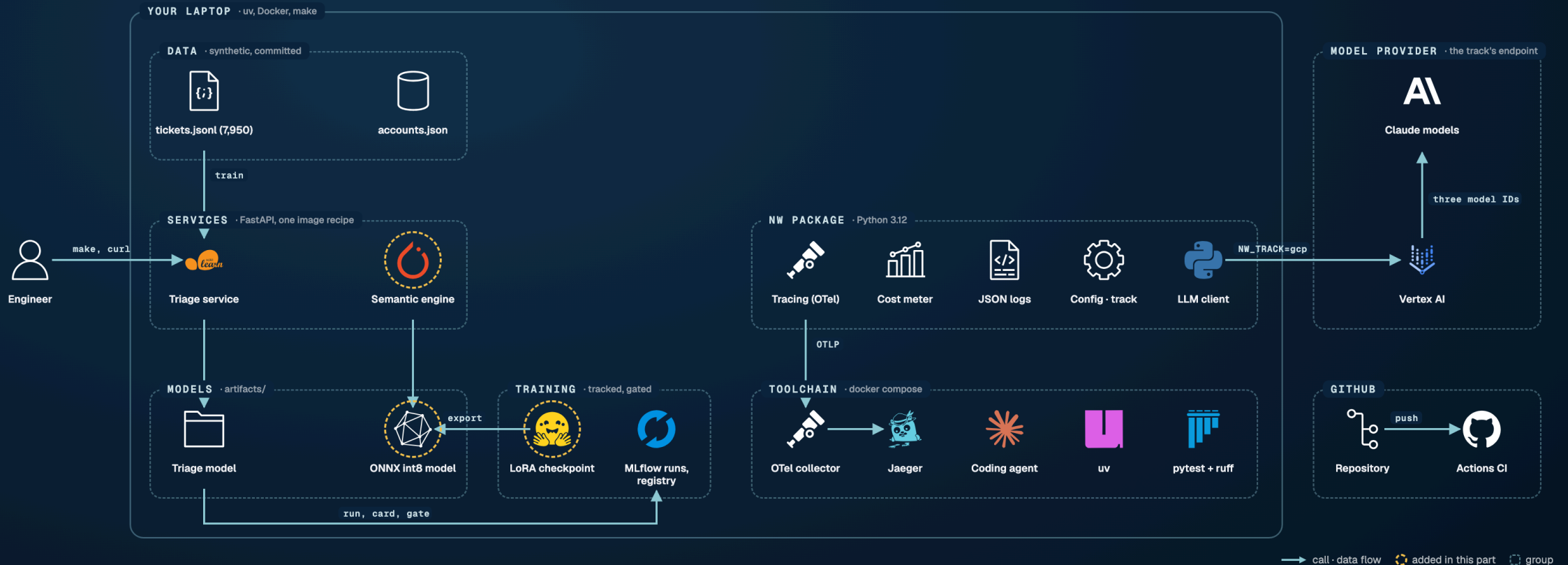
What exists after today, AWS track



The semantic engine beside the triage service, the ONNX int8 model it serves from the promoted version, and the LoRA checkpoint it was exported from, with Colab as the training fallback. Ringed: what this project adds.

What exists after today, Google Cloud track

DEEP LEARNING AND REPRESENTATION · GOOGLE CLOUD



Same engine, same graph, same checkpoint; Vertex AI is the endpoint. Nothing in Project 2 calls a model API.

Acceptance criteria

1. Trainable parameters under 2 percent, printed; the data passed its contract
2. Tags and priority clearly above chance on validation; the test evaluation by language in the metadata
3. Training resumes from a mid-run checkpoint with optimiser state
4. ONNX fp32 matches PyTorch within $1e-3$; int8 is smaller; the benchmark has measured latency for all four rows
5. The candidate passed the gate: `latest`, the alias, the summary, the card and the log agree

28 passed

what `make session03` prints when you are done

Questions worth asking

- Project 2 is slower than Project 1 by a factor you measured. For which tickets is that factor worth paying?
- The German tickets are five percent of the data and the base model is English. Where do they land in the error analysis, and what would you change?
- The gate holds int8 to three points against fp32. At what point would you serve the fp32 graph instead, and who pays for the latency?
- If the customer's stack is TensorFlow Serving, what in this session transfers unchanged?